# Project Scheduling and Time Management

Schedule management turns the WBS into a sequence of activities with durations, dependencies and dates that the team can commit to.

PMBOK Guide 7th ed. | ProjectLibre 1.9 | Scrum Guide 2020

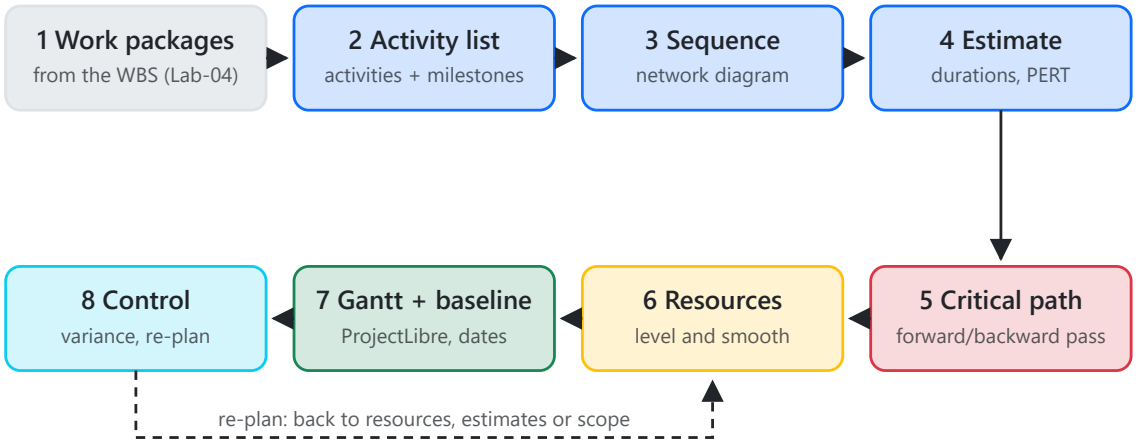
AGENDA

# What we will cover

1. Defining activities from work packages and milestones
2. Sequencing: precedence relationships (FS, SS, FF, SF), leads and lags
3. Network diagrams and the precedence diagramming method
4. Estimating durations: expert judgement, analogous, parametric, three-point (PERT)
5. Critical path method: forward pass, backward pass, float
6. Resource estimating, resource levelling and smoothing
7. Gantt charts and milestone charts in ProjectLibre or MS Project
8. Schedule compression: crashing and fast-tracking revisited
9. Agile scheduling: velocity, sprint planning, release burndown and burnup
10. Controlling the schedule: baselines, variance, re-planning
11. Chapter Quiz
12. Lab-05

Click any item to jump directly to that topic.

# From the WBS to a schedule the team can commit to



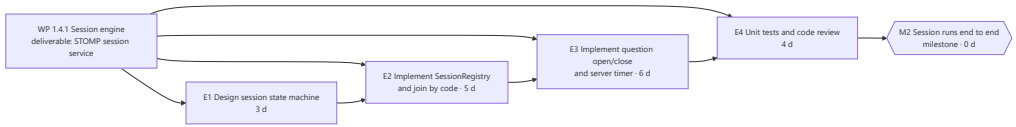

agile teams run the same loop per sprint: backlog, sprint plan, velocity, burndown

The running example of this chapter is a **teaching subset** of the Realtime Quiz project: ten activities, four students (S1 to S4), 5 working days per week, semester of 14 weeks (70 working days), pilot with 50 students in week 14.

| Id | Activity | Work package | Days |
|---|---|---|---|
| A | Write charter | 1.1.1 | 4 |
| B | Build WBS and plan | 1.1.2 | 5 |
| C | Design DB schema | 1.2.2 | 6 |
| D | Build auth API | 1.2.3 | 10 |
| E | Build session engine | 1.4.1 | 18 |
| F | Build WebSocket client | 1.4.2 | 8 |
| G | Build host console | 1.4.3 | 9 |
| H | Build student play page | 1.4.4 | 7 |
| I | Test and harden | 1.6.1 | 10 |
| J | Run pilot (50 students) | 1.6.3 | 3 |

Every technique in this chapter is worked on these ten activities, so you can check each number by hand.

# Defining activities: from one work package to the work that produces it




- **Work package** (Lab-04): the smallest *deliverable* in the WBS, a noun ("Session engine").
- **Activity**: a unit of *work* with a duration, a verb phrase ("Implement SessionRegistry"). Several activities produce one work package; an activity never spans two packages.
- **Milestone**: a significant event with zero duration ("Session runs end to end"). Use it for gates, hand-overs and sponsor decisions.
- **Size rule**: 8 to 80 hours per activity (the 8/80 rule); one owner; a testable result.
- **Rolling wave**: detail the next 4 to 6 weeks; keep later work packages as single summary activities and refine them later.

```
Activity attributes (one row of the activity list)
  id           E2
  name         Implement SessionRegistry and join by code
  work package 1.4.1 Session engine
  owner        S1
  predecessor  E1 (FS, lag 0)
  duration     5 d   effort 40 h   resource S1 100 %
  result       POST /api/sessions/{code}/join returns 200
               and the lobby lists the participant
```

# The activity list and the milestone list of the Realtime Quiz subset

```
id,name,work_package,predecessors,relationship,lag_days,duration_days
A,Write charter,1.1.1,,,,4
B,Build WBS and plan,1.1.2,A,FS,0,5
C,Design DB schema and migrations,1.2.2,B,FS,0,6
D,Build auth API (register login JWT),1.2.3,C,FS,0,10
E,Build session engine (STOMP timer),1.4.1,C,FS,0,18
F,Build WebSocket client library,1.4.2,E,FS,0,8
G,Build host console page,1.4.3,"D,F",FS,0,9
H,Build student play page,1.4.4,"D,F",FS,0,7
I,Test and harden (100 simulated users),1.6.1,"G,H",FS,0,10
J,Run pilot with 50 students,1.6.3,I,FS,5,3
```

Predecessor cells with two ids are quoted so the CSV stays one column. The 5-day lag before J is the schedule reserve (Topic 7).

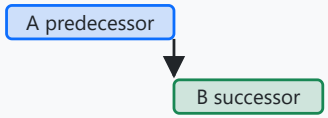
| Milestone | Criterion | Week |
|---|---|---|
| M1 Plan approved | Sponsor signs charter and baseline | 2 |
| M2 Session runs end to end | Host opens a question, a phone answers, timer closes it | 9 |
| M3 Pilot done | 50 students played, results exported | 14 |

⚠ An activity without a predecessor (except the first) or without a successor (except the last) is **dangling**: the critical path calculation will silently ignore it.

💡 The full Lab-05 list has 18 to 25 activities; this deck keeps ten so every calculation fits on one slide.

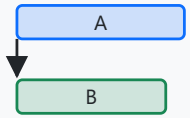
# Sequencing: four precedence relationships, leads and lags

### FS · finish to start



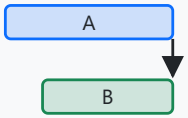

B starts when A finishes
C DB schema -> D Auth API
default: 90 % of all links

### SS · start to start



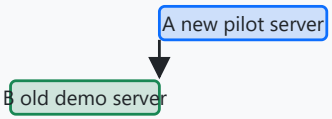

B may start when A starts
E engine SS+8 d -> F client
(fast-track option, Topic 8)

### FF · finish to finish



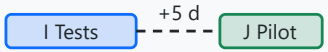

B finishes when A finishes
I tests -> user guide:
guide done when tests done

### SF · start to finish (rare)




B finishes when A starts
old server retired only once
the new one is up

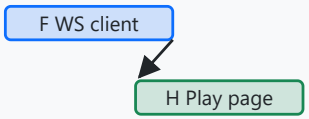
### Lag · FS + 5 d




waiting time inserted
between finish and start

here: schedule reserve,
documented in the plan,
never a hidden buffer

### Lead · FS - 2 d




successor starts 2 d before
the predecessor ends: H
codes against the mocked API

| Notation | Meaning |
|---|---|
| `C` or `3FS` | finish-to-start from task 3, no lag |
| `5SS+8d` | start 8 days after task 5 starts |
| `9FS+5d` | start 5 days after task 9 finishes (lag) |
| `6FS-2d` | start 2 days before task 6 finishes (lead) |
| `7FF` | finish together with task 7 |

- Use **FS** unless you can write down *why* the work can overlap.
- A **lead** is negative lag: it compresses; it also creates rework risk.
- ProjectLibre and MS Project use exactly this predecessor syntax (PMBOK Guide 7th ed. calls it the precedence diagramming method).

⚠ Never use a lag to model work ("2 days for the sponsor to review"): make it an activity, so it has an owner and can slip visibly.

TOPIC 2

# Four kinds of dependency and how they appear in the predecessor table

| Kind | Meaning | Realtime Quiz example |
|---|---|---|
| **Mandatory** (hard logic) | Physically or legally required | Schema (C) before auth API (D): the User table must exist |
| **Discretionary** (soft logic) | Team preference, best practice | Host console (G) before play page (H)? No: the team chose to build them in parallel |
| **External** | Outside the team's control | IT department provisions the Linode VPS before the pilot |
| **Internal** | Inside the team's control | Code review by S2 before merging the session engine |

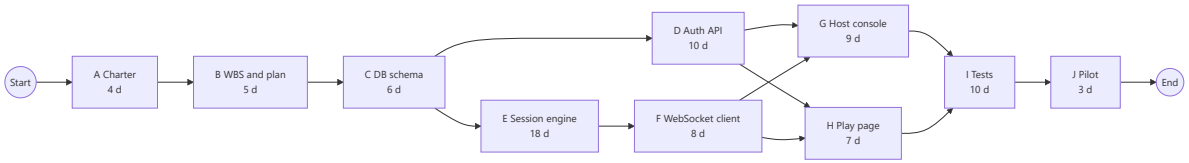
💡 Discretionary links are the first thing to remove when you must compress the schedule (Topic 8). Mark them, so you can find them later.

dependencies.csv

```
successor,predecessor,kind,relationship,lag_days,reason
B,A,mandatory,FS,0,plan needs the approved scope of the charter
C,B,mandatory,FS,0,schema follows the WBS deliverables
D,C,mandatory,FS,0,User table must exist for register/login
E,C,mandatory,FS,0,Session and Answer tables must exist
F,E,discretionary,FS,0,client coded against a finished STOMP API
F,E,discretionary,SS,8,ALTERNATIVE: start on the agreed contract
G,D,mandatory,FS,0,console needs login and JWT
G,F,mandatory,FS,0,console uses the client library
H,D,mandatory,FS,0,play page needs join + JWT
H,F,mandatory,FS,0,play page uses the client library
I,G,mandatory,FS,0,end-to-end tests need both pages
I,H,mandatory,FS,0,end-to-end tests need both pages
J,I,mandatory,FS,5,pilot after tests; 5 d schedule reserve
J,VPS,external,FS,0,VPS provisioned by the IT department
```

One row per link. The two rows for F list the baseline choice and the alternative; only one of them is active in the network.

# The precedence network: activities on nodes, dependencies on arrows




Activity-on-node (AON) is what PDM, ProjectLibre and MS Project use. The older activity-on-arrow (AOA) notation with dummy activities is only of historical interest.

```
flowchart LR
  S((Start)) --> A["A Charter<br/>4 d"]
  A --> B["B WBS and plan<br/>5 d"]
  B --> C["C DB schema<br/>6 d"]
  C --> D["D Auth API<br/>10 d"]
  C --> E["E Session engine<br/>18 d"]
  E --> F["F WebSocket client<br/>8 d"]
  D --> G["G Host console<br/>9 d"]
  F --> G
  D --> H["H Play page<br/>7 d"]
  F --> H
  G --> I["I Tests<br/>10 d"]
  H --> I
  I --> J["J Pilot<br/>3 d"]
  J --> Z((End))
```

- One **Start** and one **End** node; every activity is on a path between them.
- No loops (a cycle means the logic is wrong) and no conditional branches: the network shows what *will* happen, not what might.
- Non-FS links and lags are written on the arrow: `E -- SS+8 --> F`.

# Reading the network: every path from Start to End, and its length

```
Path 1: A-B-C-D-G-I-J   4 + 5 + 6 + 10 + 9 + 10 + 3      = 47 d
Path 2: A-B-C-D-H-I-J   4 + 5 + 6 + 10 + 7 + 10 + 3      = 45 d
Path 3: A-B-C-E-F-G-I-J 4 + 5 + 6 + 18 + 8 + 9 + 10 + 3  = 63 d  <-- longest
Path 4: A-B-C-E-F-H-I-J 4 + 5 + 6 + 18 + 8 + 7 + 10 + 3  = 61 d  (near-critical)

Project duration        = length of the longest path      = 63 working days
Semester                = 14 weeks x 5 days               = 70 working days
Slack of the project    = 70 - 63                         = 7 days
Path 4 can absorb       = 63 - 61                         = 2 days before it
                          becomes critical (H has 2 days of float, Topic 5)
```

- Enumerating paths works for ten activities; for 50 it does not. The **critical path method** (Topic 5) finds the longest path with two passes over the network, whatever its size.
- A **near-critical** path (Path 4, 2 days shorter) deserves the same attention as the critical one: one slip and it takes over.
- Merge points (G and H both feed I) are where delays combine: I cannot start before the *later* of its predecessors.

💡 Check your diagram with three questions: Is there exactly one Start and one End? Does every activity have at least one predecessor and one successor? Can you follow every path without going backwards?

# Estimating durations: four techniques on the same activity

| Technique | How | Worked example | Result |
|---|---|---|---|
| **Expert judgement** | Ask who has done it | S1 built a chat server last year: "about three and a half weeks for the session engine" | E ≈ 18 d |
| **Analogous** (top-down) | Scale a similar past activity | Last cohort: 16 d for a simpler engine; ours adds a server timer: 16 × 1.1 | E ≈ 18 d |
| **Parametric** | Rate × quantity | Auth API: 5 endpoints × 1.5 d + 2.5 d security tests | D = 10 d |
| **Bottom-up** | Sum the sub-activities | E1 + E2 + E3 + E4 = 3 + 5 + 6 + 4 | E = 18 d |
| **Three-point (PERT)** | Optimistic, most likely, pessimistic | O 14, M 18, P 28: (14 + 4×18 + 28) / 6 | E = 19.0 d |

⚠ Effort is not duration. 288 h of effort with two people at 8 h/day is 18 days; with one person at 4 h/day (a student with classes) it is 72 days. Always state the calendar and the availability.

```
Duration = Effort / (people x hours per day)

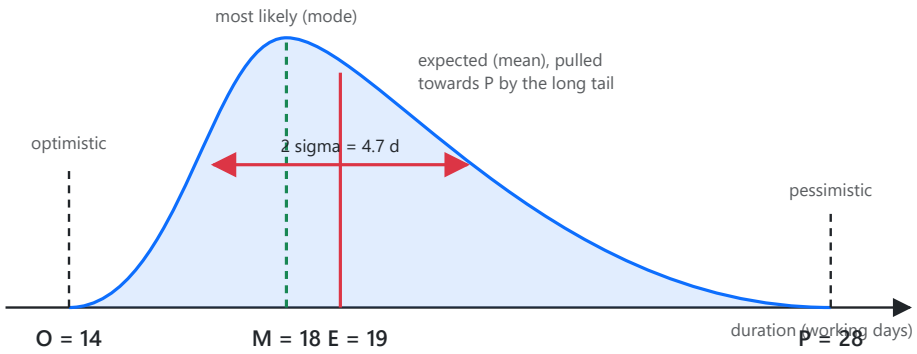
E Session engine
  effort       = 2 people x 18 d x 8 h = 288 h
  duration     = 288 / (2 x 8)         = 18 d

Same effort, one student, 4 h/day
  duration     = 288 / (1 x 4)         = 72 d

Accuracy improves with information
  charter    (week 1): rough order  -25 % to +75 %
  after WBS  (week 2): budgetary    -10 % to +25 %
  after E1   (week 4): definitive    -5 % to +10 %
```

PMBOK Guide 7th ed. lists these under "estimating"; the same techniques give the cost estimates of Chapter 06.

# Three-point estimate: the PERT formula and what the curve means




sigma = (P - O) / 6: the whole O..P range is taken as six standard deviations

```
PERT (beta) expected duration and standard deviation

  E     = (O + 4M + P) / 6
  sigma = (P - O) / 6
  var   = sigma^2

E Session engine: O = 14, M = 18, P = 28
  E     = (14 + 4 x 18 + 28) / 6 = (14 + 72 + 28) / 6
        = 114 / 6 = 19.0 d
  sigma = (28 - 14) / 6 = 14 / 6 = 2.33 d
  var   = 2.33^2 = 5.44

Triangular alternative (no weighting of M)
  E     = (O + M + P) / 3 = (14 + 18 + 28) / 3 = 20.0 d
```

- Ask for O and P as "the best and worst you have seen for similar work", not "M minus and plus a bit".
- Plan with E, not M: the most-likely value is *below* the mean whenever the pessimistic tail is long (which in software it always is).
- Standard deviations do not add; **variances add** along a path (next slide).

# PERT for all ten activities, and the probability of finishing by day 70

```
Act   O    M    P   E=(O+4M+P)/6   sigma=(P-O)/6   var
A     3    4    5       4.00           0.33        0.11
B     4    5    8       5.33           0.67        0.44
C     4    6   10       6.33           1.00        1.00
D     8   10   15      10.50           1.17        1.36
E    14   18   28      19.00           2.33        5.44
F     6    8   12       8.33           1.00        1.00
G     7    9   13       9.33           1.00        1.00
H     5    7   10       7.17           0.83        0.69
I     8   10   16      10.67           1.33        1.78
J     2    3    5       3.17           0.50        0.25

Critical path A-B-C-E-F-G-I-J (found with M durations, Topic 5)
  expected length = 4.00+5.33+6.33+19.00+8.33+9.33+10.67+3.17 = 66.2 d
  path variance   = 0.11+0.44+1.00+5.44+1.00+1.00+1.78+0.25  = 11.0
  path sigma      = sqrt(11.0)                               =  3.3 d
```

```
Probability of finishing by a target T
  Z = (T - E_path) / sigma_path, then read P(Z)

T = 70 d (end of week 14, pilot deadline)
  Z = (70 - 66.2) / 3.3 = 1.15  ->  P = 87 %

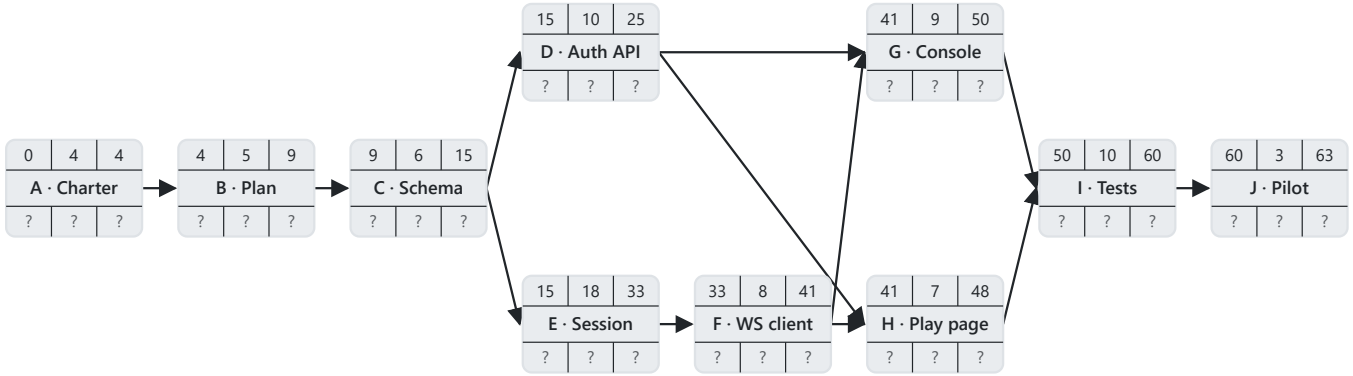
T = 63 d (the most-likely plan)
  Z = (63 - 66.2) / 3.3 = -0.97 ->  P = 17 %

Z table (one-sided):
  Z  -1.0  -0.5   0.0   0.5   1.0   1.5   2.0
  P   16 %  31 %  50 %  69 %  84 %  93 %  98 %
```

ⓘ A schedule built from most-likely durations has a **17 % chance** of being met. The 7 days between day 63 and day 70 are not luxury: they are the reserve that lifts the probability to 87 %.

# Critical path method, step 1: the forward pass gives early dates

cell layout: ES | Dur | EF over LS | TF | LF (day numbers, 0 = start of day 1)




forward pass: ES = max(EF of predecessors), EF = ES + duration; project finishes on day 63

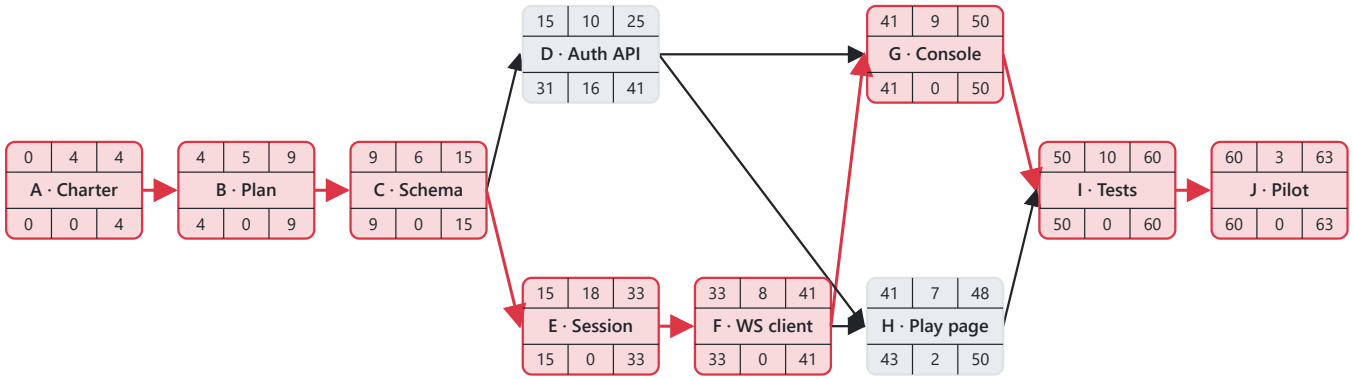
```
ES = max(EF of all predecessors)
EF = ES + duration
Start activity: ES = 0

A: ES 0,  EF 0 + 4  = 4
B: ES 4,  EF 4 + 5  = 9
C: ES 9,  EF 9 + 6  = 15
D: ES 15, EF 15 + 10 = 25
E: ES 15, EF 15 + 18 = 33
F: ES 33, EF 33 + 8  = 41
G: ES max(25, 41) = 41, EF 50
H: ES max(25, 41) = 41, EF 48
I: ES max(50, 48) = 50, EF 60
J: ES 60, EF 60 + 3  = 63
```

Convention: day numbers count elapsed working days, so "EF 4" means the end of day 4. The 1-based convention (ES 1, EF 4, next ES 5) gives the same floats; pick one and state it.

# Step 2: the backward pass gives late dates, float and the critical path

cell layout: ES | Dur | EF over LS | TF | LF (day numbers, 0 = start of day 1)




red boxes and arrows: critical path A-B-C-E-F-G-I-J = 63 working days, total float 0

```
LF = min(LS of all successors)
LS = LF - duration
End activity: LF = project end = 63

J: LF 63, LS 63 - 3  = 60
I: LF 60, LS 60 - 10 = 50
G: LF 50, LS 50 - 9  = 41
H: LF 50, LS 50 - 7  = 43
F: LF min(41, 43) = 41, LS 33
D: LF min(41, 43) = 41, LS 31
E: LF 33, LS 33 - 18 = 15
C: LF min(31, 15) = 15, LS 9
B: LF 9,  LS 4
A: LF 4,  LS 0

Total float TF = LS - ES = LF - EF
Critical: TF = 0 -> A B C E F G I J
```

# Float table: how much each activity may slip, and whom it would disturb

```
Act  Dur  ES  EF  LS  LF   TF = LS-ES   FF = min(ES succ) - EF   Critical?
A     4   0   4   0   4       0            4 - 4   = 0           yes
B     5   4   9   4   9       0            9 - 9   = 0           yes
C     6   9  15   9  15       0           15 - 15  = 0           yes
D    10  15  25  31  41      16       min(41,41) - 25 = 16       no
E    18  15  33  15  33       0           33 - 33  = 0           yes
F     8  33  41  33  41       0       min(41,41) - 41 = 0        yes
G     9  41  50  41  50       0           50 - 50  = 0           yes
H     7  41  48  43  50       2           50 - 48  = 2           no
I    10  50  60  50  60       0           60 - 60  = 0           yes
J     3  60  63  60  63       0           63 - 63  = 0           yes (end)

Critical path: A-B-C-E-F-G-I-J = 63 d.  Near-critical: H (TF 2).
D may slip 16 days without moving the end date (TF) and without
delaying G or H (FF), because G and H wait for F anyway.
```

| Term | Meaning | Who cares |
|---|---|---|
| **Total float** | Delay possible without moving the project end | Project manager |
| **Free float** | Delay possible without moving any successor's ES | The next activity's owner |
| **Critical path** | Longest path; every activity has TF = 0 | Everyone |
| **Negative float** | Imposed finish date earlier than EF: the plan is already late | Sponsor |

💡 Float belongs to the path, not to the person: if D uses 16 days of float, nothing else on that path may slip. Levelling (Topic 6) spends float deliberately.

⚠ Free float is never larger than total float. If your table says otherwise, an ES is wrong.

# Resource estimating: who does what, and where the calendar breaks

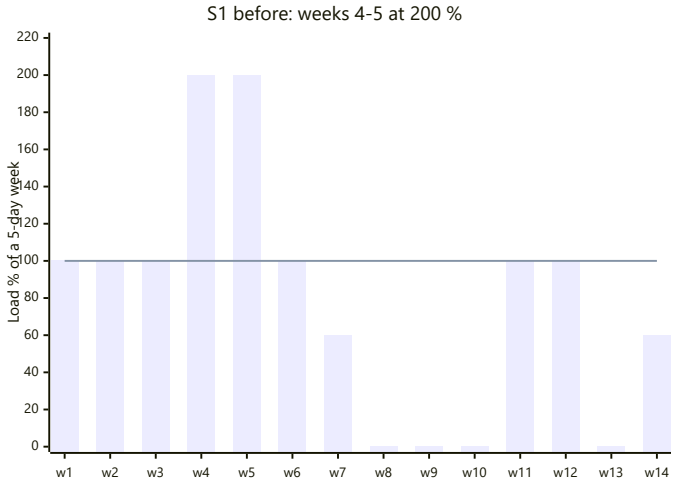
| Act | Days | Resources (100 % each) | Effort h | Window (days) |
|---|---|---|---|---|
| A | 4 | S1, S2 | 64 | 0 to 4 |
| B | 5 | S1, S2, S3, S4 | 160 | 4 to 9 |
| C | 6 | S1 | 48 | 9 to 15 |
| D | 10 | S1 | 80 | 15 to 25 |
| E | 18 | S1, S2 | 288 | 15 to 33 |
| F | 8 | S3 | 64 | 33 to 41 |
| G | 9 | S3 | 72 | 41 to 50 |
| H | 7 | S4 | 56 | 41 to 48 |
| I | 10 | S1, S2, S3, S4 | 320 | 50 to 60 |
| J | 3 | S1, S2, S3, S4 | 96 | 60 to 63 |

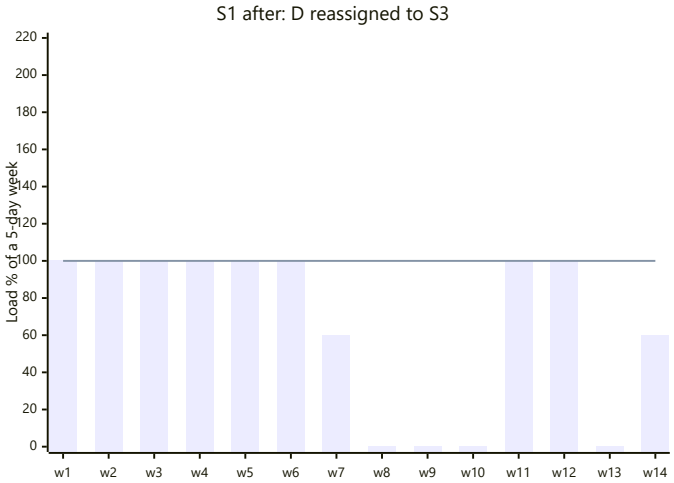
Effort = people × days × 8 h. Total 1 248 h; this feeds the budget of Lab-06. S4 works on the quiz-authoring packages (not in the subset) in weeks 3 to 8.

- **Resource estimating**: for each activity, which skill, how many people, what percentage, from which calendar (a student is not available 40 h a week during exams).
- **Over-allocation**: S1 is assigned to D *and* E from day 15 to 25: 200 % of one person. The CPM did not notice, because CPM assumes unlimited resources.
- **Resource levelling**: change start dates (and sometimes the end date) so nobody exceeds 100 %. Uses float first; may extend the critical path.
- **Resource smoothing**: adjust only inside the available float; the end date never moves; some over-allocation may remain.
- Teams level *per person*: the team histogram (max 20 person-days per week) looked fine while S1 was at 200 %.

⚠ ProjectLibre's automatic levelling delays tasks; it never reassigns people. The reassignment decision is yours (next slide).

# Resource histogram of S1 before and after levelling, and the decision behind it



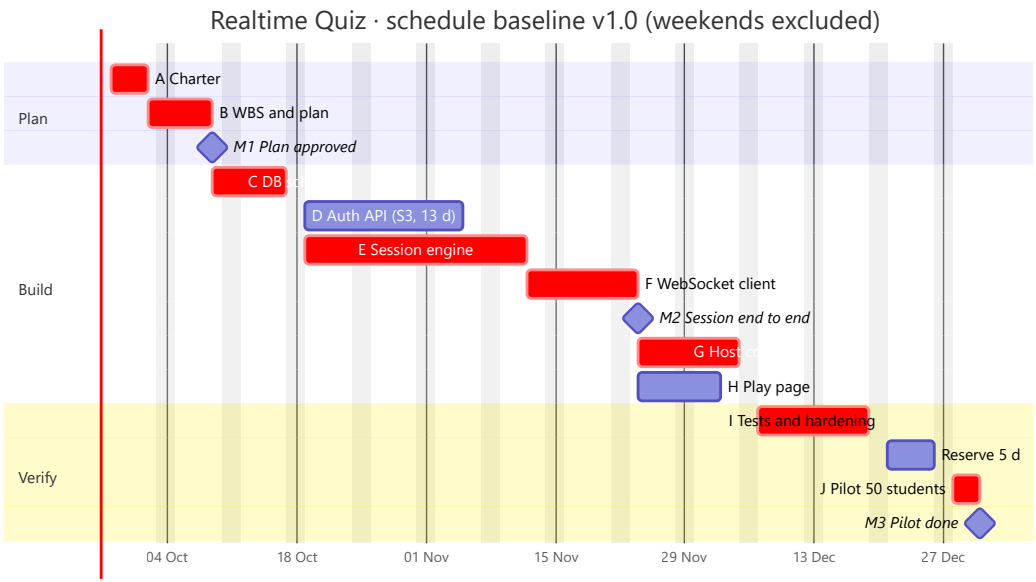

```
Conflict: S1 on D (10 d) and E (18 d)
from day 15. E is critical, D has
TF = 16 d. Three options:

1 Delay D until S1 is free (day 33)
  D: 33 -> 43, but LF(D) = 41
  end date slips 2 d (63 -> 65)
2 Reassign D to S3 (idle days 15-33)
  S3 is new to Spring Security:
  duration 10 -> 13 d, D: 15 -> 28
  28 <= LF 41: no slip, +24 h effort
3 Split D: 5 d now, 5 d after E
  D ends day 38 <= 41: no slip,
  but two context switches for S1

Decision: option 2 (levelling by
reassignment). Recorded in the
schedule management plan.
```

Red line = capacity (100 %). Weeks 13 and 14 hold the reserve and the pilot (Topic 7). Automatic levelling in ProjectLibre would have chosen option 1 and moved the end date.

# The Gantt chart: the levelled schedule on the calendar, with milestones and reserve




```
gantt
  title Realtime Quiz · baseline v1.0
  dateFormat YYYY-MM-DD
  axisFormat %d %b
  excludes weekends
  section Plan
  A Charter        :crit, a, 2026-09-28, 4d
  B WBS and plan   :crit, b, after a, 5d
  M1 Plan approved :milestone, m1, after b, 0d
  section Build
  C DB schema      :crit, c, after b, 6d
  D Auth API       :d, after c, 13d
  E Session engine :crit, e, after c, 18d
  ...
  I Tests          :crit, i, after g h, 10d
  Reserve 5 d      :r, after i, 5d
  J Pilot          :crit, j, after r, 3d
```

- Week 1 starts Mon 28 Sep 2026; the pilot lands on 28 to 30 Dec, week 14, exactly as the charter promised.
- The **reserve** is a visible task owned by the PM: day 63 to 68 raises the finish probability from 17 % to about 71 % (Z = 0.54).
- A **milestone chart** is this Gantt with only the diamonds: the sponsor's view.

# Building the same schedule in ProjectLibre (MS Project works the same way)

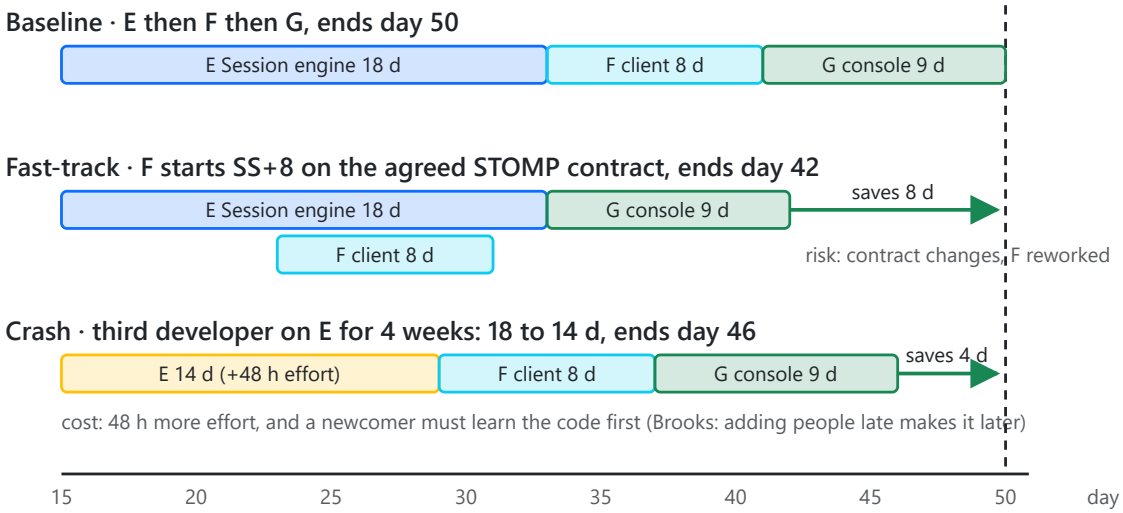
```
1  File > New: name "Realtime Quiz", start 2026-09-28,
   "Forward scheduled" (dates are computed from the start)
2  Project > Calendar: Standard, Mon-Fri 8 h; mark holidays and
   the exam week as non-working; students at 100 % of that calendar
3  Gantt view: type Name and Duration for each row (4d, 5d, ...);
   Indent rows under summary tasks 1 Plan, 2 Build, 3 Verify;
   a milestone is a task with Duration 0d
4  Predecessors column: "3" means 3FS; "5SS+8d" and "9FS+5d" add
   the relationship and the lag; leads are negative: "6FS-2d"
5  Resources view: S1..S4 as Work resources, Max units 100 %,
   Lecturer 10 %; Task > Information > Resources to assign
6  View > Resource usage (histogram): red bars = over-allocation
7  Resources > Level resources: "within available slack" first;
   if a red bar remains, reassign by hand and re-check the end date
8  Task > Set baseline (Baseline 0) once the sponsor approves
9  View > Network diagram to check the critical path (red boxes);
   View > Tracking Gantt to compare progress with the baseline
10 File > Save as lab05/schedule.pod; File > Print to PDF or PNG
```

| You want to see | ProjectLibre view | MS Project |
|---|---|---|
| Bars on a calendar | Gantt | Gantt Chart |
| ES/EF/LS/LF, float | Gantt, add columns Early Start, Late Finish, Total Slack | Schedule table |
| Critical path | Network diagram (red) | Network Diagram |
| Over-allocation | Resource usage, Resource histogram | Resource Graph |
| Baseline vs actual | Tracking Gantt | Tracking Gantt |

💡 Type durations in `d` (working days) and let the tool place the dates. Typing dates by hand removes the logic, and the critical path disappears.

⚠ ProjectLibre shows *Total Slack* for total float; a negative value means a constraint ("Must finish on") is already violated.

# Schedule compression: fast-tracking buys time with risk, crashing buys time with money




| | Fast-tracking | Crashing |
|---|---|---|
| What | Run sequential activities in parallel (FS becomes SS+lag, or a lead) | Add resources, overtime or better tools to critical activities |
| Costs | Rework risk, more coordination | Money or effort; diminishing returns |
| Where | Discretionary links on the critical path | Critical activities with the lowest cost per day saved |
| Limit | Mandatory logic cannot overlap | The crash duration; after it, a new path becomes critical |

Compress only the critical path. Shortening D (16 days of float) saves nothing. Re-run the CPM after every step: the critical path moves.

# Crashing worked example: cheapest day first, and watch the second path

```
Crash slope = (crash effort - normal effort) / (normal d - crash d)

Act  Normal d  Crash d  Normal h  Crash h  Slope h/day   Max saving
C        6        4        48       78     30/2 = 15        2 d
E       18       14       288      336     48/4 = 12        4 d
F        8        6        64       84     20/2 = 10        2 d
G        9        7        72       96     24/2 = 12        2 d
I       10        7       320      380     60/3 = 20        3 d

Goal: pilot 5 days earlier (63 -> 58 d). Critical path A-B-C-E-F-G-I-J.
Step 1  F  -2 d (cheapest, 10 h/d)   cost  20 h   duration 61
        check path 4 (A-B-C-E-F-H-I-J): also -2 -> 59, still shorter
Step 2  E  -3 d of 4 (12 h/d)        cost  36 h   duration 58   done
        path 4 -> 56: G stays critical, H keeps 2 d float
Total: 5 d saved for 56 h  (average 11.2 h per day saved)

Not chosen: G -2 d costs 24 h but after 2 d H (TF 2) becomes critical
too, so the 3rd day would need G and H together: 12 + 8 = 20 h/d.
```

- Crash the critical activity with the **lowest slope** first, one day at a time, up to its maximum saving.
- After each step recompute every path: when a second path becomes critical, further days must be bought on *both* paths.
- Stop when the target is reached or when the slope exceeds what a day is worth to the sponsor (Chapter 06 puts a price on it).
- Fast-tracking first, crashing second is the usual order: the SS+8 link on F would have saved 8 days for 0 h, at the price of rework risk.

⚠ Crashing tests (I) is the most expensive option and the most tempting one, because tests come last. Cutting tests instead of crashing them is a scope change, not a schedule technique.

# Agile scheduling: velocity turns a backlog into a sprint count, not a date

```
Velocity = story points completed per sprint (done = done)
Backlog of the Realtime Quiz MVP: 120 points, 2-week sprints
Observed velocity of the team: 20 to 25 points per sprint

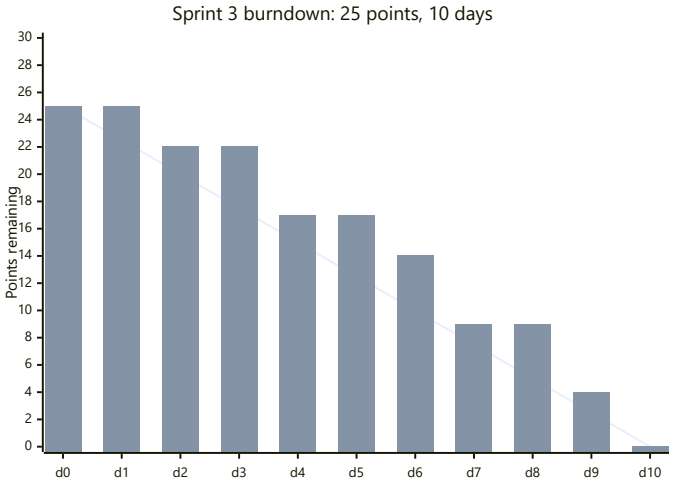
Sprints needed = backlog / velocity
  optimistic    120 / 25 = 4.8  -> 5 sprints -> 10 weeks
  pessimistic   120 / 20 = 6.0  -> 6 sprints -> 12 weeks
Sprint 0 (weeks 1-2: charter, WBS, environment) + 5 or 6 sprints
  -> MVP done in week 12 (best) to week 14 (worst)
  -> the week-14 pilot holds only if velocity never drops below 20

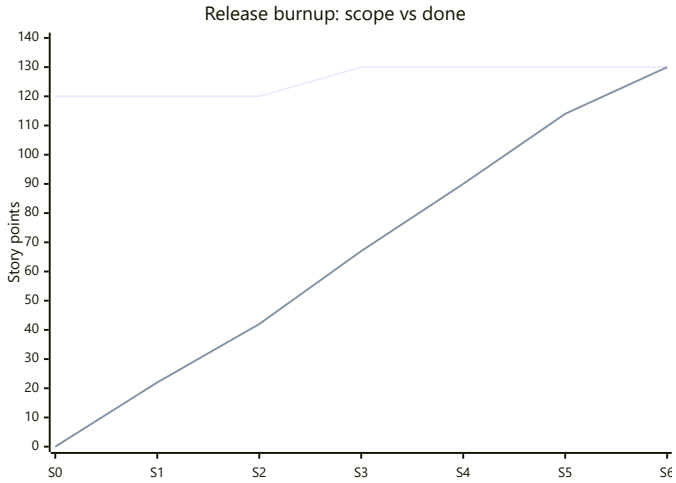
Sprint planning (Scrum Guide 2020): capacity first, then commit
  capacity   = 4 students x 10 d x 4 h = 160 h  (half days, classes)
  focus 75 % = 120 h of sprint work -> about 24 points at 5 h/point
  commit 22-25 points, never the optimistic 30
```

| Sprint | Weeks | Goal (activities of the subset) | Pts |
|---|---|---|---|
| 0 | 1-2 | Charter, WBS, plan, repo (A, B) | - |
| 1 | 3-4 | Schema, register and login (C, D) | 22 |
| 2 | 5-6 | Join by code, lobby (E1, E2) | 20 |
| 3 | 7-8 | Question open/close, timer, client (E3, E4, F) | 25 |
| 4 | 9-10 | Host console, play page (G, H) | 23 |
| 5 | 11-12 | Tests, hardening, load test (I) | 24 |
| 6 | 13-14 | Reserve, pilot with 50 students (J) | 16 |

- The sprint plan and the Gantt describe the *same* work: sprints are time-boxes, the network still says what can start.
- Forecast with a **range** (5 to 6 sprints), and update it every sprint from the real velocity.
- PMI's Agile Practice Guide calls this hybrid: predictive milestones outside, iterations inside.

# Sprint burndown (inside a sprint) and release burnup (across sprints)



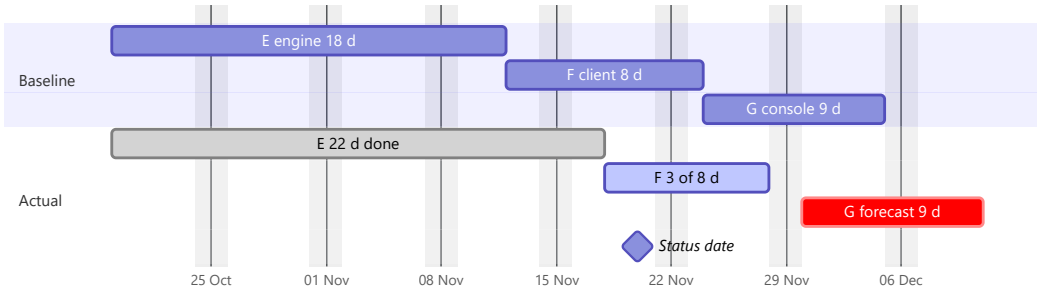

```
Reading the burndown
  ideal line: 25 / 10 = 2.5 pts per day
  bars above the line = behind
  flat days (d1, d3, d5, d8): stories
  only count when "done", so the
  chart drops in steps, not slopes

Reading the burnup (after sprint 4)
  scope   130 (was 120: +10 in S3,
          media in questions added)
  done     90  (22 + 20 + 25 + 23)
  velocity avg = 90 / 4 = 22.5
  remaining = 130 - 90 = 40
  sprints  = 40 / 22.5 = 1.8 -> 2
  forecast: done end of sprint 6
  at velocity 20: 2.0 -> still S6
  at velocity 18: 2.2 -> S7: late
```

A burndown hides scope changes (the line just goes up); the burnup shows them as a separate line. For release forecasts prefer the burnup.

# Controlling the schedule: the baseline is the ruler, the variance is the reading

Week 8 status (Fri 20 Nov): baseline vs actual for the Build phase




- **Schedule baseline**: the approved version of the schedule (dates, durations, logic), frozen when the sponsor signs M1. Changed only through integrated change control (Chapter 04).
- **Status date**: the day you measure. Everything left of it is actual; right of it is forecast.
- **Schedule variance in days** = baseline finish − forecast finish. Here E finished 4 days late (timer race conditions), so F, G and everything after them forecast 4 days late.
- Chapter 06 adds the earned value form: SV = EV − PV in currency and SPI = EV / PV.

⚠ Do not "fix" a variance by editing the baseline. The baseline records what was promised; the forecast records what will happen. Both must be visible.

# The schedule variance report and the four ways to re-plan

```
Schedule status report        status date: Fri 20 Nov 2026 (week 8)
Baseline: v1.0 approved 9 Oct   Critical path: A-B-C-E-F-G-I-J

Act  Baseline finish  Actual / forecast   Var   Status  Cause
E    Wed 11 Nov       Tue 17 Nov (actual)  -4 d  done    timer races
F    Mon 23 Nov       Fri 27 Nov           -4 d  3/8 d   started late
G    Fri 04 Dec       Thu 10 Dec           -4 d  not st. inherits
I    Fri 18 Dec       Thu 24 Dec           -4 d  not st. inherits
Reserve: 5 d planned, 4 d consumed, 1 d left
J    Wed 30 Dec       Wed 30 Dec            0 d  fixed   pilot date holds

Trend: 4 d lost in 8 weeks on the critical path; 1 d reserve left
for 6 weeks of remaining work. Probability of the 30 Dec pilot
without action: Z = (68 - 70.2) / 3.3 = -0.67 -> about 25 %.

Proposed action (decision needed by Mon 23 Nov):
  fast-track H: start on the mocked client 2 d early (FS-2d)   +2 d
  drop "media in questions" from I's test scope to Could       +1 d
```

| Re-planning option | Changes | Needs |
|---|---|---|
| Use reserve | Nothing, until the reserve is gone | PM decision |
| Compress | Fast-track or crash remaining critical work | Risk or effort |
| Reduce scope | Move Could items after the pilot | Change request |
| Re-baseline | New dates become the promise | Sponsor approval; keep the old baseline for lessons learned |

A good status report ends with a decision request, a date for the decision and the cost of not deciding. Red without a proposal is just bad news.

WRAP-UP

# Best practices and common mistakes in scheduling

### ✓ Estimate with the team

The person who will do the work gives O, M and P. Estimates handed down from the PM are commitments nobody made.

### ✓ One visible reserve

Size the reserve from the path sigma (here 5 days for about 70 %), own it as PM, and report how much is left every week.

### ✓ Level per person, then re-run the CPM

After every levelling or compression step the critical path can move. The float table is only valid for the network it was computed on.

### ✗ Durations typed as dates

Entering start and finish dates by hand removes the logic. The tool cannot compute float, and every change becomes manual work.

### ✗ Padding every estimate

Ten activities each padded by 20 % make a 63-day plan into 76 days and still leave the critical activity unprotected. Padding hides; a reserve protects.

### ✗ Students at 100 %

A student has classes, other labs and exams. Model 4 h/day or 50 % units, or every sprint will "fail" for a reason the plan created.

# Chapter quiz 10 questions

**1. Three-point estimate for the session engine: O = 14, M = 18, P = 28 days. What is the PERT expected duration?**

- 18.0 days
- 19.0 days
- 20.0 days
- 21.0 days

**2. Activity D (Auth API) has ES = 15 and LS = 31. What is its total float?**

- 0 days
- 10 days
- 16 days
- 46 days

**3. "The user guide may finish no earlier than 2 days after the tests finish." Which relationship models this?**

- FS + 2 d
- SS + 2 d
- FF + 2 d
- SF − 2 d

**4. Which statement about the critical path is correct?**

- It is the shortest path through the network
- Every activity on it has zero total float
- It never changes once computed
- It contains only mandatory dependencies

**5. Activity H finishes on day 48 (EF) and its only successor I has ES = 50. What is the free float of H?**

- 0 days
- 2 days
- 7 days
- 50 days

**6. Which compression technique shortens the schedule by adding effort or cost to critical activities?**

- Fast-tracking
- Crashing
- Resource smoothing
- Rolling wave planning

**7. The MVP backlog holds 120 story points and the team completes 24 points per sprint. How many sprints does the release forecast need?**

- 4
- 5
- 6
- 24

**8. What distinguishes resource smoothing from resource levelling?**

- Smoothing may extend the project end date, levelling never does
- Smoothing adjusts only within the available float, so the end date does not move
- Smoothing reassigns people, levelling changes durations
- There is no difference

**9. The critical path has an expected length of 66.2 days and a standard deviation of 3.3 days. The probability of finishing within 70 days is about**

- 17 %
- 50 %
- 87 %
- 99 %

**10. During execution the session engine finishes 4 days late. What should the project manager do with the schedule baseline?**

- Move the baseline finish dates by 4 days so the variance disappears
- Keep the baseline, record the variance and the forecast, and propose actions
- Delete the baseline and set a new one every week
- Change the baseline only for non-critical activities

WRAP-UP

# Summary

| | | |
|---|---|---|
| Work packages are deliverables; activities are the verb phrases that produce them; milestones have zero duration. | Forward pass gives ES and EF, backward pass gives LS and LF; total float = LS − ES; the critical path has zero float. | Crashing buys days with effort (cheapest slope first); fast-tracking buys days with rework risk; both only on the critical path. |
| FS is the default link; SS, FF, SF, leads and lags each need a written reason, and a lag never models work. | Free float is what an activity can use without disturbing its successors; near-critical paths deserve the same attention. | Velocity turns a backlog into a range of sprints (120 points at 20 to 25 is 5 to 6 sprints), updated every sprint from the burnup. |
| PERT: E = (O + 4M + P) / 6 and sigma = (P − O) / 6; add variances, not sigmas, along a path. | Level per person: reassign or delay inside float before you extend the end date; re-run the CPM afterwards. | Baseline once; measure variance against it with a status date; re-plan with options; re-baseline only through change control. |

Open Lab-05: 5 tasks + 1 challenge

Next chapter: 06 · Cost and Budget Management